

Professor Matheus – Guia Introdutório ao MAUI

1. Introdução ao .NET MAUI

O .NET MAUI (Multi-platform App UI) é um toolkit de interface de usuário (UI) que permite a criação de layouts nativos compartilhados entre iOS, macOS, Android e Windows a partir de uma única base de código. Seu principal propósito é fornecer uma solução abrangente para aplicações empresariais, reduzindo o custo total de propriedade ao permitir o compartilhamento de código de lógica e interface entre todas as plataformas de destino.

Estrutura Básica de um Projeto MAUI

Diferente de versões anteriores como o Xamarin.Forms, o .NET MAUI utiliza um projeto único que pode atingir múltiplos destinos, eliminando a necessidade de projetos separados para cada plataforma. A organização típica de pastas inclui:

- Models: Classes de dados.
- Views: Páginas da aplicação (geralmente em XAML).
- ViewModels: Lógica de apresentação e estado.
- Services: Interfaces e classes de acesso a dados ou serviços.

a. O Padrão MVVM (Model-View-ViewModel)

O padrão MVVM é essencial no desenvolvimento MAUI para separar a lógica de negócios e de apresentação da interface do usuário (UI). Essa separação torna a aplicação mais fácil de testar, manter e evoluir.

O Papel de cada Camada

1. Model: São classes não visuais que encapsulam os dados da aplicação, incluindo o modelo de domínio, lógica de negócios e validação. Exemplos incluem objetos de transferência de dados (DTOs) e objetos de entidade.
2. View: Responsável por definir a estrutura, o layout e a aparência do que o usuário vê na tela. Idealmente, a View é definida em XAML com um code-behind limitado que não contém lógica de negócios.
3. ViewModel: Atua como o intermediário. Ela implementa propriedades e comandos aos quais a View pode se conectar via Data Binding. A ViewModel coordena as interações da View com as classes do Model e notifica a View sobre mudanças de estado através de eventos de notificação.

Separação de Responsabilidades e Vantagens

No MVVM, a View "conhece" a ViewModel e a ViewModel "conhece" o Model, mas o Model não conhece a ViewModel e a ViewModel não conhece a View. Isso isola a interface das mudanças no modelo de dados. As principais vantagens incluem:

- Testabilidade: Desenvolvedores podem criar testes unitários para a ViewModel e o Model sem precisar da View.
- Manutenibilidade: A UI pode ser redesenhada sem tocar no código da lógica, desde que as propriedades de ligação permaneçam consistentes.
- Colaboração: Designers podem focar na View enquanto desenvolvedores trabalham na lógica da ViewModel simultaneamente.

b. Data Binding e Notificações de Mudança

O Data Binding é o mecanismo que conecta as propriedades da View às propriedades da ViewModel. Para que a View seja atualizada automaticamente quando um valor na ViewModel muda, a ViewModel deve implementar a interface `INotifyPropertyChanged`.

Exemplo Prático com MVVM Toolkit: O uso do `CommunityToolkit.Mvvm` simplifica essa implementação através do `ObservableObject`.

```
// Exemplo de ViewModel usando Source Generators do Toolkit
public partial class LoginViewModel : ObservableObject
{
    [ObservableProperty]
    private string _userName; // Gera automaticamente a propriedade UserName com notificação
}
```

Na View (XAML), a ligação é feita assim:

```
<Entry Text="{Binding UserName, Mode=TwoWay}" />
```

Isso garante que qualquer texto digitado no Entry atualize a propriedade na ViewModel e vice-versa.

c. Comandos (Commands)

Em vez de usar manipuladores de eventos (como `Clicked`) no code-behind, o MVVM utiliza Commands. Comandos encapsulam a ação a ser executada e a mantêm desacoplada da representação visual.

Implementação de Comando: A interface `ICommand` define os métodos. e.g. `Execute` (a ação em si) e `CanExecute` (se a ação pode ocorrer).

```
public ICommand LoginCommand { get; }

public LoginViewModel()
{
    // RelayCommand é uma implementação comum no MVVM Toolkit
    LoginCommand = new RelayCommand(ExecuteLogin);
}

private void ExecuteLogin()
{
    // Lógica de login aqui
}
```

No XAML, o botão é vinculado ao comando:

```
<Button Text="Entrar" Command="{Binding LoginCommand}" />
```

d. Navegação no MAUI

O .NET MAUI utiliza o componente Shell como hospedeiro de navegação. A navegação é baseada em rotas, o que facilita o gerenciamento de páginas.

Como navegar via ViewModel: Uma prática comum é criar um serviço de navegação que pode ser invocado pela ViewModel para manter a lógica testável.

```
// Navegando para uma rota registrada  
await Shell.Current.GoToAsync("//Main/Catalog");
```

Passagem de Parâmetros

Para passar dados entre páginas, utilizam-se parâmetros de consulta (query parameters). A ViewModel de destino captura esses dados usando o atributo QueryProperty.

```
[QueryProperty(nameof(OrderId), "OrderId")]  
public class OrderDetailViewModel : ViewModelBase  
{  
    public string OrderId { get; set; } // Recebe o valor automaticamente da rota  
}
```

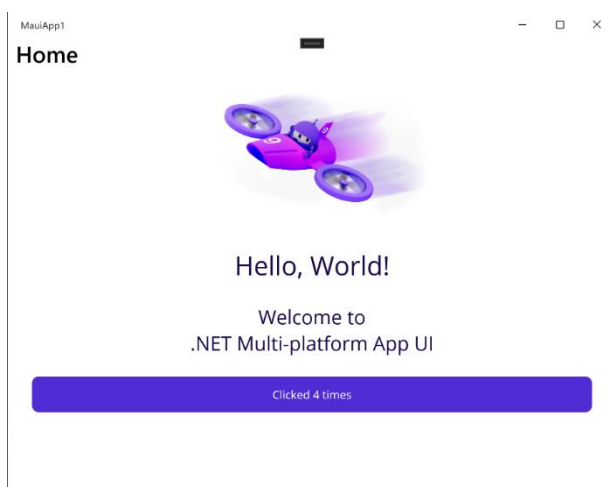
2. Criando nosso primeiro projeto e primeira página.

a. Criando o template MAUI App

Abra o Visual Studio versão 2022 ou superior, não confundir com o VS Code, depois clique em “Create a new Project”. Na página seguinte, selecione o template .NET MAUI App e clique em “Next”, após isso altere o nome do projeto, caso desejado, e escolha o local onde o projeto será salvo. Verifique se o Framework utilizado é o .NET 10.0 LTS.

Após o template ser criado, rode o projeto em Windows Machine e confirme se o template está funcionando.

Figura 1 MAUI Template App



Fonte: MAUI Template App

b. Criando nossa primeira página.

Criação da página

Pare de rodar a aplicação, caso esteja aberta, na sequência, clique com o botão esquerdo do mouse em cima do nome de seu projeto, sendo o padrão “MauiApp1”.

Vá em: Add Item → New Item

Pesquise por XAML no local de busca e selecione o layout “ContentPage”. Nomeie sua página com atenção, pois, renomeações em ambiente MAUI são complexas, sendo usualmente mais fácil a criação novamente do elemento com o nome correto.

Ponteiro para acessar página criada

Pronto, agora temos a página criada. Na sequência iremos criar o ponteiro para que possamos acessar essa página. Para isso iremos acessar o código de “AppShell.xaml” e adicionar um novo ShellContent, possibilitando a navegação entre páginas.

```
<ShellContent
    Title="Nova Página"
    ContentTemplate="{DataTemplate local:MinhaNovaPagina}" />
```

Com isso, teremos uma nova opção para acessar a página criada.

Altere title para o título desejado, o texto a ser exibido ao usuário na hora de escolher a página a ser acessada.

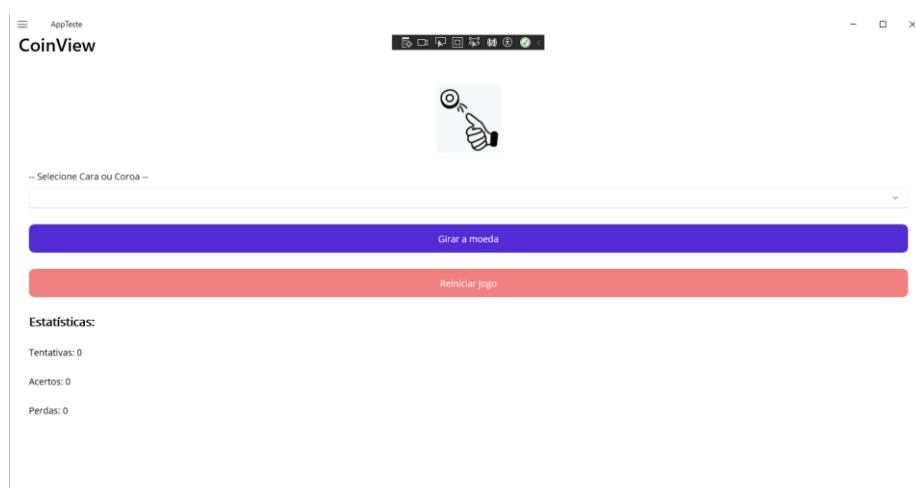
Altere ContentTemplate com o ponteiro da localização da página, sendo necessário inserir o nome da página que criamos após o texto “local:”.

```
ContentTemplate="{DataTemplate local:InsiraAquiONomeDaPaginaCriada}" />
```

3. Criando o aplicativo “Cara ou Coroa”

Iremos criar o seguinte aplicativo:

Figura 2 CoinApp



Fonte: Elaborada pelo autor

a. Criando a estrutura no Padrão MVVM (Model-View-ViewModel)

Vamos criar nossa estrutura de pastas/diretórios em nosso projeto. Para isso clicamos com o botão direito do mouse no nome de nossa solução e selecionamos “Add” → “New Folder”. Iremos criar três novas pastas: “Models”, “ViewModels” e “Views”.

b. Criação da View

Prosseguindo iremos construir nossa View. Clicamos com o botão direito na pasta View e vamos em “Add” → “New Item...”, nessa etapa vamos em “Show all templates”, pesquisamos por XAML e selecionamos o template “ContentPage”. Nomearemos como CoinView.

Dentro de VerticalStackLayout iremos adicionar os seguintes itens:

- Image (coinLogo.png)
- Picker (Opção para selecionar cara ou coroa)
- Button (Botão para girar a moeda)
- Button (Botão para reiniciar o jogo)
- Label (Texto: Estatísticas)
- Label (Texto: Tentativas)
- Label (Texto: Acertos)
- Label (Texto: Perdas)
- Label (Resultado)
- Image (Imagem do resultado, coinCara.png ou coinCoroa.png)

c. Criação do ponteiro para acessarmos nossa página

Com a página criada, o próximo passo é realizar a criação do ponteiro no arquivo AppShell.xaml para que possamos acessar nossa página. Para isso, iremos adicionar no shell inicial o xmlns indicando o caminho de nossa página, da seguinte forma:

```
xmlns:viewCoin="clr-namespace:NomeDoSeuProjeto.Views"
```

Isso é necessário porque a página não está mais no local padrão do aplicativo, estando na subpasta View, sendo necessário a criação desse ponteiro. Na sequência iremos criar um novo ShellContent informando que teremos uma nova página e iremos indicar seu caminho.

```
<ShellContent  
  Title="Cara ou Coroa"  
  ContentTemplate="{DataTemplate viewCoin:CoinView}"  
>
```

Com isso já podemos abrir nosso aplicativo e abrir nossa nova página.

d. Criação das classes C# e preparação

Começamos com a criação de nosso back-end, o código por trás da página. Iremos criar uma classe na pasta Models chamada Coin.cs, para isso vamos na pasta Models → Add → Class...

Para facilitar a operação e manipulação das variáveis dessa classe, iremos trocar de *internal* para *public*, de forma a deixar a classe pública externamente, podendo sempre que quisermos acessar essa.

```
public class Coin
```

Na sequência iremos criar uma classe na pasta ViewModels chamada CoinViewModel.cs, para isso vamos na pasta ViewModels → Add → Class...

Antes de iniciarmos nosso código, iremos criar os ponteiros para as pastas Models e Views. Usamos o parâmetro *using* para isso, seguido do nome do projeto e a pasta a ser referenciada, separada por um ponto. Ficará da seguinte forma:

```
using NomeDoSeuProjeto.Models;  
using NomeDoSeuProjeto.Views;
```

Além desses ponteiros de pastas a classe deverá herdar conteúdo do framework MAUI. Iremos utilizar alguns comandos do Windows e o comando Model do MAUI, ficará da seguinte forma:

```
using System.Windows.Input;  
using CommunityToolkit.Mvvm.ComponentModel;
```

Caso apareça um erro na linha “*using CommunityToolkit.Mvvm.ComponentModel;*”, significa que o pacote *CommunityToolkit.Mvvm* não está instalado. Para instalar, clicamos com o botão direito em nosso projeto → Manage Nuget Packages... → Buscamos por “*CommunityToolkit.Mvvm*” em “Browse” → Seleccionamos o pacote e clicamos em “Install” → Apply → I Accept.

Quando aparecer “===== Finished =====” no terminal, significa que o pacote foi instalado. Ao retornar na classe, observamos que não há mais erros.

e. Manipulação de conteúdos com C# - Coin.cs

Iniciaremos nosso código na classe Coin.cs, adicionando nossas principais funções. Primeiramente iremos criar uma variável que irá armazenar o lado que será escolhido. Chamaremos essa variável de “Lado” e inicialmente ela será vazia.

```
public string Lado { get; set; } = string.Empty
```

Para criarmos uma sequência que irá escolher o lado aleatoriamente e automaticamente, solicitaremos a função *Random()*, gravando numa *string* o nome do lado escolhido. Incluiremos essa string dentro da função *SortearLado* da seguinte forma.

```
public string SortearLado()  
{  
    int ladoSorteado = new Random().Next(2);  
    Lado = (ladoSorteado == 0) ? “Cara” : “Coroa”;  
    return Lado;  
}
```

Por fim iremos criar o código para gerar o resultado do jogo. Incluiremos esse código na função *Jogar(ladoEscolhido)*. Essa função irá coletar o lado que o usuário selecionou, irá buscar a função

SortearLado para verificar qual lado será escolhido, realizar a comparação de ambos e informar o resultado numa string.

```
public string Jogar(string ladoEscolhido)
{
    string ladoSorteado = SortearLado();
    string resultado = (Lado == ladoEscolhido) ?
        $"Parabéns, você pediu {ladoEscolhido} e deu {Lado}." :
        $"Que pena, você pediu {ladoEscolhido} e deu {Lado}.";
    return resultado;
}
```

Com essa variável e as duas funções criadas, concluímos a criação do código de nosso Model em Coin.cs. A próxima etapa será construir o backend de CoinViewModel.cs, que irá fazer a ponte e interligar a escolha do usuário com a solicitação e escolha aleatória do sistema.

f. Manipulação de conteúdos com C# - CoinViewModel.cs

Transformaremos a classe CoinViewModel adicionando o parâmetro ObservableObject, ficará da seguinte forma:

```
public partial class CoinViewModel : ObservableObject
```

Na sequência iremos ler o conteúdo de nossa View, da seguinte forma.

```
private readonly CoinView _view;
```

Antes de continuarmos a manipulação de conteúdo, iremos introduzir o jogador em nossa página, jogando um alerta de boas-vindas. Para isso criaremos nossa classe principal, que irá receber como objeto nossa View.

```
public CoinViewModel(CoinView view)
```

Na sequência damos as boas vindas com um alerta, no MAUI utilizamos DisplayAlert para alerta um conteúdo que irá ficar acima de nossa tela. O DisplayAlert possui três conteúdos, o título da caixa de alerta, a mensagem da caixa de alerta e o botão de encerramento da caixa de alerta. O código ficará da seguinte forma.

```
Application.Current.MainPage.DisplayAlert("Mensagem", "Bem-vindo(a) ao jogo Cara ou Coroa!", "Começar agora!");
```

Com isso já estamos quase terminando de preparar nosso primeiro código do back-end! Para que o alerta funcione, precisamos enviar nossa view para a classe CoinViewModel. Realizamos esse envio pelo CoinView.xaml.cs, o back-end direto de nossa view. Uma única linha irá fazer a mágica acontecer! Após a inicialização do componente adicionamos o envio de nossa view para o ViewModel.

```
this.BindingContext = new CoinViewModel(this);
```

Ao rodar nosso projeto, e mudarmos para a tela do jogo “Cara ou Coroa”, conseguiremos visualizar nosso alerta.

REFERÊNCIAS

Build your first app. Disponível em: <https://learn.microsoft.com/en-us/dotnet/maui/get-started/first-app?view=net-maui-10.0&tabs=vswin&pivots=devices-android>

Create a .NET MAUI APP. Disponível em: <https://learn.microsoft.com/en-us/dotnet/maui/tutorials/notes-app?view=net-maui-10.0>

Enterprise Application Patterns Using .NET MAUI. Disponível em: <https://aka.ms/maui-ebook>